Надо сделать: Дать ответ на задание **Надо сделать:** Получить оценку

Срок сдачи: воскресенье, 1 ноября 2026, 23:59

TL;DR

Часть I. Нейронные сети для табличных данных по учебным примерам — до 10 баллов

- Подготовка данных + baseline — **2**
- Две модели из учебных примеров — **2**
- Гиперпараметры — **3**
- Сравнение, ошибки, importance, ручные объекты — **3**

> Одна нейросеть → максимум **7/10**

Часть II. Нейронные сети для табличных данных — самостоятельное исследование архитектуры — до 15 баллов

- Подготовка данных под новую архитектуру — **2**
- Новая архитектура / архитектуры — **4**
- Исследование и оптимизация — **6**
- Итоговое сравнение + ручные объекты — **3**

> Одна новая архитектура → максимум **10/15**

Подробная инструкция - после нажатия на **"Добавить ответ на задание"**

Нейронные сети для табличных данных

I. Нейронные сети для табличных данных по учебным примерам — до 10 баллов

1.1. Выберите набор данных

Выберите набор данных для задачи **классификации** или **регрессии**.

Классификация — целевая переменная принимает конечное число классов.
Регрессия — целевая переменная числовая и может принимать большое количество значений.

В идеале используйте один из наборов данных, который вы уже «крутили» ранее.

Древнее зло пробудилось.

Также разрешается использовать данные:

- из вашей проектной работы;
- из вашей рабочей деятельности, если этими данными можно делиться;
- из открытых источников: OpenML, Kaggle и т. п.

Нельзя использовать данные из соревнования Kaggle текущего семестра.

Ориентиры для набора данных:

- от **5 содержательных признаков** без учёта целевой переменной;
- от **1000 объектов**, желательно больше;

- желательно наличие как числовых, так и категориальных признаков.

Примеры допустимых наборов данных:

- Прогнозирование оттока клиентов: https://www.kaggle.com/competitions/customer-churn-prediction-2020
- Кредитный скоринг: https://www.openml.org/search?type=data&sort=runs&status=active&id=31
- Предсказание цены дома: https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques

Можно выбрать и любой другой подходящий набор данных.

1.2. Предварительная обработка и baseline — до 2 баллов

Выполните адекватную предварительную обработку данных.

Если вы уже выполняли EDA и preprocessing этого набора данных в предыдущей лабораторной работе — отлично. Можно сделать ссылку на предыдущую работу и показать только итоговый pipeline и необходимые результаты.

Необходимо разделить данные как минимум на:

train / validation / test

При этом:

- preprocessing обучается **только на train**;
- `train` используется для обучения модели;
- `validation` используется для выбора архитектуры, эпох и гиперпараметров;
- `test` используется только для итоговой оценки.

Не используйте test для выбора параметров модели.

Добавьте хотя бы один классический tabular baseline, например:

- CatBoost;
- LightGBM;
- XGBoost;
- либо сильную модель, полученную вами в предыдущих лабораторных работах.

Все сравниваемые модели должны использовать **одно и то же разбиение данных** и одинаковые итоговые метрики.

Для нейросетей обратите внимание на масштаб числовых признаков и целевой переменной. Если это необходимо для выбранной модели, выполните стандартизацию target, обучив преобразование только на train.

1.3. Выберите две нейросетевые архитектуры из учебных примеров — до 2 баллов

В учебных блокнотах рассмотрены:

- **MLP / tabular ResNet** — считаются одним семейством;
- **FT-Transformer**;
- **TabNet**;
- **TabTransformer**;
- **GANDALF**.

Выберите **две архитектуры**.

Желательно выбирать модели, основанные на разных идеях. Например:

- ResNet + FT-Transformer;
- TabNet + TabTransformer;
- FT-Transformer + GANDALF;
- TabNet + GANDALF.

Сначала запустите модели в разумной базовой конфигурации — условно «из коробки».

Для каждой модели зафиксируйте:

- основные параметры архитектуры;
- количество эпох;
- validation-метрику;
- желательно время обучения;
- при желании — количество обучаемых параметров.

Важно про FAST_MODE

В учебных блокнотах присутствует режим:

```
FAST_MODE = True
```

Он предназначен **только для проверки того, что notebook запускается и pipeline работает**.

В FAST-режиме используется меньше данных и/или эпох, поэтому качество моделей может быть очень плохим.

Метрики из FAST_MODE нельзя использовать для итоговых выводов и сравнения архитектур.

Для содержательных экспериментов переключите:

```
FAST_MODE = False
```

1.4. Исследуйте и оптимизируйте выбранные модели — до 3 баллов

Попробуйте улучшить выбранные архитектуры.

Не требуется запускать огромный Grid Search или AutoML на сотни экспериментов.

Гораздо важнее показать, что вы понимаете:

> какой параметр меняете → зачем его меняете → какой результат получили.

Например, можно исследовать следующие параметры.

MLP / ResNet

- количество блоков;
- размер скрытых слоёв;
- dropout;
- learning rate;
- weight decay.

FT-Transformer

- количество Transformer-блоков;
- размер token embedding;
- количество attention heads;
- dropout;
- learning rate.

TabNet

- `n_d`;
- `n_a`;
- `n_steps`;
- `gamma`;
- `lambda_sparse`;
- размер embeddings;
- learning rate.

TabTransformer

- размер embeddings;
- количество attention heads;
- количество attention blocks;
- dropout;
- learning rate.

GANDALF

- количество GFLU stages;
- dropout;
- начальная sparsity;
- learnable sparsity;
- learning rate.

Покажите несколько разумных экспериментов и кратко прокомментируйте результаты.

Отдельный плюс в карму — исследование **unsupervised / self-supervised pretraining**, если выбранная архитектура это поддерживает.

1.5. Сравните модели и исследуйте их поведение — до 3 баллов

Сравните:

- классический tabular baseline;
- первую выбранную нейросеть;
- вторую выбранную нейросеть.

Недостаточно написать:

> Модель A дала $R^2 = 0.82$, модель B — 0.79, поэтому A лучше.

Ожидается хотя бы небольшой анализ:

- итоговых метрик;
- распределения ошибок;
- крупных ошибок;
- возможных причин различий между моделями;
- поведения моделей на разных объектах.

Для регрессии можно использовать, например:

- RMSE;
- MAE;
- R^2;
- `actual vs predicted`;
- распределение абсолютных ошибок.

Для классификации выбирайте метрики в соответствии с особенностями задачи и распределением классов.

Также желательно исследовать **feature importance или её аналоги**.

Для честного сравнения разных архитектур рекомендуется использовать хотя бы один общий model-agnostic метод, например:

Permutation Importance

Если модель имеет собственные механизмы интерпретации — например masks у TabNet — покажите их дополнительно и сравните с общим методом.

Обязательная проверка на новых данных, созданных вручную

Создайте вручную **не менее 10 новых объектов**, которых нет в исходном наборе данных.

Получите для них предсказания:

- классического baseline;
- первой нейросети;
- второй нейросети.

Новые объекты должны проходить **тот же preprocessing pipeline**, что и реальные данные.

Не кодируйте категории и не нормализуйте признаки вручную отдельно от основного pipeline.

Желательно создать несколько разных типов объектов:

- обычный объект, похожий на объекты обучающей выборки;
- несколько похожих объектов, в которых осмысленно изменяется один или несколько признаков;
- необычный, но возможный объект;
- при наличии категориальных признаков — объект с категорией, которой не было в train.

Например, для задачи оценки стоимости автомобиля можно создать несколько почти одинаковых автомобилей:

```
Toyota Prius, 2008 год, пробег 280 000 км
Toyota Prius, 2014 год, пробег 150 000 км
Toyota Prius, 2020 год, пробег  50 000 км
```

При этом остальные признаки оставить одинаковыми.

Получится небольшой **counterfactual sanity check**: можно посмотреть, насколько разумно модель реагирует на изменение года выпуска и пробега.

Важно:

созданные вручную данные не являются дополнительной test-выборкой.

У таких объектов обычно нет известного правильного target, поэтому по ним нельзя оценивать RMSE, R^2 и т. п.

Это проверка цепочки:

```
новый объект
      ↓
preprocessing
      ↓
model
      ↓
prediction
```

Прокомментируйте:

- выглядит ли prediction правдоподобно;
- меняется ли он ожидаемым образом;
- насколько расходятся разные модели;
- что произошло с неизвестной категорией;
- находится ли объект примерно в области обучающих данных;
- каким predictions стоит доверять меньше.

Результаты на придуманных объектах **нельзя использовать для подбора гиперпараметров**.

Правдоподобный prediction не доказывает качество модели.
Неправдоподобный prediction — хороший повод проверить модель и pipeline.

Ограничение по баллам

Если выполнена только **одна нейросетевая архитектура**, максимальная оценка за часть I — **7 баллов**.

II. Нейронные сети для табличных данных — самостоятельное исследование

архитектуры — до 15 баллов

Теперь задача немного меняется.

В первой части вы работали с архитектурами, для которых были предоставлены учебные примеры.

Во второй части необходимо **самостоятельно найти и подключить архитектуру, которой среди учебных блокнотов нет**.

Основная задача здесь — не переписать статью с нуля, а разобраться:

```
paper / документация
        ↓
реализация
        ↓
API
        ↓
preprocessing
        ↓
эксперимент
        ↓
сравнение с уже известными моделями
```

2.1. Данные и экспериментальный протокол

Предпочтительно использовать **тот же dataset**, что и в части I.

Древнее зло пробудилось ×2.

Также желательно сохранить:

- то же разбиение `train / validation / test`;
- те же метрики;
- тот же классический baseline;
- те же созданные вручную объекты.

В таком случае вы действительно сравниваете **архитектуры**, а не результаты двух разных экспериментов.

Другой dataset использовать можно, но желательно объяснить, зачем он понадобился.

2.2. Подготовка данных и baseline — до 2 баллов

Если используется тот же набор данных, повторять полный EDA не требуется.

Достаточно сослаться на первую часть работы и показать:

- итоговый набор признаков;
- preprocessing;
- размеры `train / validation / test`;
- baseline;
- особенности preprocessing, которые пришлось изменить для новой архитектуры.

Если выбранная модель требует другого представления категориальных или числовых признаков — обязательно опишите это.

2.3. Выберите одну или две новые архитектуры — до 4 баллов

Нельзя использовать архитектуры, для которых уже предоставлены учебные блокноты:

- MLP / ResNet;
- FT-Transformer;
- TabNet;
- TabTransformer;
- GANDALF.

Можно рассмотреть, например:

- **TabM**;
- **AutoInt**;
- **NODE**;
- **GATE**;
- **TabR**;
- другие современные архитектуры для табличных данных.

Также на лекции обсуждается **SAINT**.

Для него готовый учебный notebook не предоставляется: доступные реализации оказались неудобны с точки зрения совместимости с современным окружением.

Использовать SAINT **можно**, если вам удалось найти рабочую реализацию, но специально заниматься археологией Python-окружения не требуется.

Как выбирать реализацию

Ищите:

- поддерживаемую библиотеку;
- официальный репозиторий авторов;
- достаточно свежую реализацию с документацией.

Оцените:

- когда репозиторий обновлялся;
- поддерживается ли используемая версия Python;
- какие зависимости нужны;
- насколько понятен API;
- есть ли examples/documentation.

Если выбранная библиотека не устанавливается из-за устаревших зависимостей:

не требуется героически оживлять окружение пятилетней давности.

Выберите другую реализацию или архитектуру и кратко напишите, почему первоначальный вариант пришлось заменить.

Это тоже нормальный инженерный результат.

Перед использованием модели коротко объясните её идею

Необходимо своими словами объяснить:

- какую проблему пытается решить архитектура;
- чем она отличается от обычного MLP;
- какую информацию она использует;
- какую роль играют её основные компоненты.

Не требуется переписывать математическое описание всей статьи.

Хочется увидеть понимание идеи модели, а не копию README.

2.4. Обучите и исследуйте выбранные модели — до 6 баллов

Сначала получите baseline выбранной архитектуры с разумными параметрами.

Зафиксируйте:

- основные параметры;
- число эпох;
- validation-метрику;
- желательно время обучения.

Затем исследуйте несколько гиперпараметров.

Не нужно искать абсолютный математический максимум.

Лучше провести небольшой осмысленный эксперимент:

> Я предположил, что изменение параметра X повлияет на Y.
> Проверил несколько значений.
> Получил Z.
> Вот как я интерпретирую результат.

Если модель показывает подозрительный результат, например:

```
R² ≈ 0 или R² < 0 не спешите делать вывод:
```

> архитектура не работает.

Сначала проверьте:

- достаточно ли эпох;
- продолжает ли уменьшаться validation loss;
- корректен ли learning rate;
- правильно ли обработаны категориальные признаки;
- нужен ли scaling числовых признаков;
- нужен ли scaling target;
- нет ли ошибки в preprocessing;
- корректно ли разделены train/validation/test.

Для сильной работы дополнительно можно:

- повторить лучшую конфигурацию на нескольких random seeds;
- показать среднее качество и разброс;
- исследовать sensitivity к параметрам;
- оценить время обучения и inference.

2.5. Итоговое сравнение — до 3 баллов

Сведите результаты всех основных моделей в одну таблицу.

Например:

Модель	Семейство	RMSE	MAE	R^2
CatBoost	Classical ML	...	...	...
FT-Transformer	Учебная NN	...	...	...
TabNet	Учебная NN	...	...	...
TabTransformer	Учебная NN	...	...	...
GANDALF	Учебная NN	...	...	...
TabM / NODE / ...	Самостоятельная	...	...	...

Не обязательно включать вообще все учебные модели — достаточно тех, которые использовались в вашей работе.

Сравните:

- итоговые метрики;
- характер ошибок;
- стабильность результата;
- сложность обучения;
- время обучения;
- интерпретируемость;
- feature importance или её аналоги.

Более новая или более сложная модель **не обязана** оказаться лучше.

Вывод:

> «Современная архитектура на выбранном наборе данных преимущества не дала»

— является абсолютно нормальным результатом, если эксперимент проведён корректно.

Повторите проверку на вручную созданных данных

Если используется тот же dataset, передайте **те же самые ручные объекты**, что и в части I, новой архитектуре.

Получится особенно интересный эксперимент:

```
один объект
    ↓
CatBoost
FT-Transformer
TabNet
TabTransformer
GANDALF
новая архитектура
    ↓
разные predictions
```

Сравните результаты и попробуйте объяснить существенные расхождения.

Если новая архитектура особенно плохо ведёт себя на необычных объектах или неизвестных категориях — это тоже полезный результат.

Ограничение по баллам

Если в самостоятельной части исследована только **одна новая архитектура**, максимальная оценка за часть II — **10 баллов**.

Для получения максимальной оценки необходимо исследовать две архитектуры либо выполнить особенно содержательное исследование одной архитектуры по согласованию с преподавателем.

Требования к оформлению notebook

Notebook должен читаться как небольшой технический отчёт:

```
Постановка задачи
        ↓
Данные
        ↓
Preprocessing
        ↓
Baseline
        ↓
Neural models
        ↓
Hyperparameter experiments
        ↓
Test
        ↓
Анализ ошибок
        ↓
Интерпретация
        ↓
Новые ручные объекты
        ↓
Выводы
```

Хочется видеть:

- понятную структуру и заголовки разных уровней;
- комментарии не только к коду, но и к полученным результатам;
- отсутствие запущенных ячеек с ошибками;

- понятные имена переменных;
- адекватно оформленные графики;
- подписи и пояснения к визуализациям;
- разумное количество рисунков;
- итоговые таблицы сравнения;
- выводы по проведённым экспериментам.

Не нужно присылать 100 графиков без комментариев.

Оставьте визуализации, которые помогают рассказать историю эксперимента.

Интерактивные Plotly-графики использовать можно и приветствуется.

Ещё раз про FAST_MODE

Если в учебном notebook используется:

```
FAST_MODE = True
```

это режим проверки работоспособности.

Перед финальным экспериментом переключите:

```
FAST_MODE = False
```

Результаты FAST_MODE в отчёт как итоговые результаты моделей не принимаются.

Что сдавать

Необходимо предоставить:

- `.ipynb` файл вашего решения;

или

- ссылку на notebook.

Если предоставляется ссылка, убедитесь, что преподавателю открыт доступ.

Перед отправкой желательно выполнить:

Runtime → Restart / Run all

или аналогичную операцию в используемой среде и убедиться, что notebook действительно выполняется **с начала до конца без ошибок**.

При сдаче задания после дедлайна - штраф в 1% за каждый просроченный день (штраф не более 50%).

Добавить ответ на задание

Ответ в виде текста

Редактировать Вид Вставить Формат Инструменты Таблица Справка

p слов - 0 Build with tinyMCE

Ответ в виде файла

Максимальный размер новых файлов: 10 Мбайт, максимальное количество прикрепленных файлов: 20

Файлы

Для загрузки файлов перетащите их сюда.

Сохранить Отмена

На портале электронного обучения размещены курсы для студентов и сотрудников УрФУ. Доступ к материалам осуществляется по корпоративной учетной записи.

Инфо

Уральский федеральный университет

Дирекция информационных технологий

Запрос в техническую поддержку

Контакты

Телефон : +7 (343) 227-20-70

Эл.почта : support@urfu.ru

Copyright © 2026 Уральский федеральный университет